Frametap

RubyFPV Frame Extraction Plugin

Technical Documentation | UplinkRobotics Internship | 2026

1. What This Project Is

r_plugin_frametap is a C++ shared library (.so) built against the official RubyFPV core plugin SDK. It listens on a local UDP socket for an MPEG-TS wrapped H.264 video stream, decodes each frame using FFmpeg, and saves JPEG images to disk alongside a structured JSON event log. The plugin was developed and fully tested in software on Ubuntu, no physical Wombat hardware was used. The video stream was simulated using ffmpeg, which transmits a local MP4 file over UDP in the same MPEG-TS format that a real RubyFPV ground station produces. Connecting to real hardware requires two small code changes documented in Section 8.

Attribute	Actual value, verified from source code
Plugin name	FrameTap - RubyFPV Frame Extractor
Plugin GUID	FT9921-RFP44-XT01-FRAME
Plugin version	1.0
Language	C++17
Build output	r_plugin_frametap.so (60KB shared library)
UDP port	7012 (localhost test)
Stream URL	udp://127.0.0.1:7012, hardcoded in source
Save interval	SAVE_EVERY_N = 1, every decoded frame is saved
Tested stream	640x360 MP4 over localhost UDP, simulated via ffmpeg
Hardware tested	No, software simulation only

2. How It Works

2.1 Plugin Lifecycle

When core_plugin_init is called by RubyFPV (or the test runner), the plugin does four things:

- Creates the frametap_frames/ output directory using mkdir -p
- Opens frametap_frames/log.json and writes the opening JSON bracket
- Sets g_bRunning = true
- Spawns a background pthread running ft_udp_thread and returns 0 immediately

When core_plugin_uninit is called, the plugin:

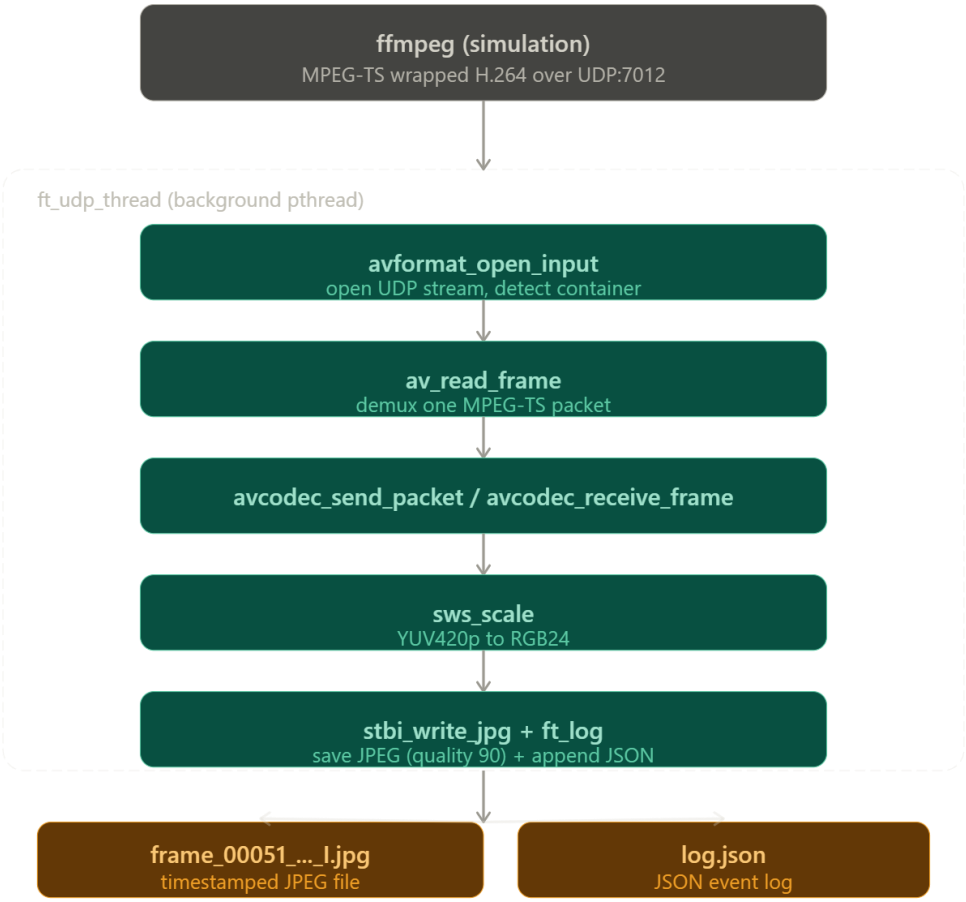
- Sets g_bRunning = false
- Calls pthread_join to wait for ft_udp_thread to finish
- Writes the closing JSON bracket and closes log.json

2.2 The Decoder Thread, ft_udp_thread

All frame extraction happens in this background thread:

- Opens udp://127.0.0.1:7012 using avformat_open_input with 1MB buffer
- Calls avformat_find_stream_info to detect the MPEG-TS video stream
- Locates the first video stream by scanning fmt_ctx->nb_streams
- Finds the H.264 decoder using avcodec_find_decoder on the stream codec_id
- Opens the decoder context with AV_CODEC_FLAG2_FAST for speed
- Enters a loop: av_read_frame -> skip non-video packets -> check keyframe flag
- Skips all packets until the first keyframe (I-frame) to ensure clean decode start
- Sends each packet to decoder: avcodec_send_packet / avcodec_receive_frame
- On first frame: creates SwsContext for YUV420p -> RGB24 conversion
- Calls sws_scale to convert pixel format
- Builds output filename with frame index, date/time, and I/P flag
- Calls stbi_write_jpg to save JPEG
- Calls ft_log to append JSON entry to log.json

2.3 Data Flow




```
ffmpeg (simulation) or RubyFPV ground station
        MPEG-TS wrapped H.264 -> UDP:7012 (test) / UDP:5600 (real)

ft_udp_thread  (background pthread)
  avformat_open_input("udp://127.0.0.1:7012")
  av_read_frame()             demux one MPEG-TS packet
  avcodec_send_packet()       feed H.264 to decoder
  avcodec_receive_frame()     get decoded YUV420p frame
  sws_scale()                 YUV420p -> RGB24
  stbi_write_jpg()            RGB24 -> JPEG file
  ft_log()                    append JSON entry

frametap_frames/frame_00051_20260331_133145_I.jpg
frametap_frames/log.json
```

3. Files

File	Description
plugin_frametap.cpp	All plugin code, SDK hooks, UDP thread, decoder, frame writer, JSON logger
stb_image_write.h	Header-only JPEG encoder, single include, no install needed
Makefile	Builds r_plugin_frametap.so with -std=c++17 -fPIC -shared
test_runner.cpp	Loads .so via dlopen, calls init, waits 30 seconds, calls uninit
test_runner	Compiled test binary
r_plugin_frametap.so	Compiled shared library, the plugin itself
frametap_frames/	Output directory, JPEG files + log.json

---

4. Build Instructions

4.1 Prerequisites

```
sudo apt install g++ make libavcodec-dev libavformat-dev libavutil-dev
libswscale-dev
clone RubyFPV and save code files in new directory:
RubyFPV/code/r_plugin_frametap
```

4.2 Build the Plugin

```
cd ~/RubyFPV/code/r_plugin_frametap
make
# Output: r_plugin_frametap.so  (60KB)
```

4.3 Makefile Contents

```
CXX      = g++
CXXFLAGS = -std=c++17 -O2 -Wall -fPIC
LIBS     = -lavcodec -lavformat -lavutil -lswscale -lpthread
TARGET   = r_plugin_frametap.so
SRC      = plugin_frametap.cpp
$(TARGET): $(SRC)
    $(CXX) $(CXXFLAGS) -shared -o $(TARGET) $(SRC) $(LIBS)
```

-fPIC generates position-independent code, required for shared libraries on Linux.

---

5. Running the Plugin

5.1 Order of Operations

Start the test runner before ffmpeg. The plugin opens a UDP socket and blocks waiting for data, if ffmpeg starts first some packets are lost before the socket is ready.

```
# Terminal 1, start plugin first
cd ~/RubyFPV/code/r_plugin_frametap
./test_runner
# Wait for: [frametap] stream opened - decoding 640x360


# Terminal 2, start simulated stream
ffmpeg -stream_loop -1 -re -i ~/test_feed.mp4 \
       -an -vcodec libx264 -f mpegts udp://127.0.0.1:7012
```

5.2 Verified Terminal Output, from actual test run on 03/31/2026

```
[test] plugin name: FrameTap - RubyFPV Frame Extractor
[test] initializing...
[frametap] plugin initialized -- saving every 1 frames to
frametap_frames
[frametap] stream opened -- decoding 640x360
[frametap] frame 0 | P-frame | 640x360
[frametap] frame 1 | P-frame | 640x360
...
[frametap] frame 51 | I-frame | 640x360
...
[frametap] frame 96 | I-frame | 640x360
...
[frametap] plugin unloaded. N frames saved.
```

Note: total frames saved depends on how long the test runner is left running before Ctrl+C or the timeout.

---

6. Verified Output, from actual test run

6.1 JPEG Files, confirmed on disk

```
frame_00000_20260331_133144_P.jpg   <- first frame, P-frame
frame_00001_20260331_133144_P.jpg
frame_00002_20260331_133144_P.jpg
frame_00003_20260331_133144_P.jpg
frame_00004_20260331_133144_P.jpg
```

```
...
frame_00051_20260331_133145_I.jpg   <- first I-frame (keyframe)
...
frame_00096_20260331_133147_I.jpg   <- second I-frame
...
frame_00108_20260331_133148_P.jpg   <- last frame in this run
```

6.2 JPEG Filename Format

```
frame_00051_20260331_133145_I.jpg
  ^^^^^   ^^^^^^^^ ^^^^^^  ^
  index   date     time    I=keyframe  P=inter-frame
```

6.3 log.json, confirmed frame type breakdown

From the actual test run:


```
[
  {
    "frame_id": 0,
    "timestamp": 1774985504000,
    "frame_type": "P-frame",
    "width": 640,
    "height": 360
  },
  {
    "frame_id": 51,
    "timestamp": 1774985505000,
    "frame_type": "I-frame",
    "width": 640,
    "height": 360
  },
  ...
]
```


Note: The timestamp field represents the time the frame was captured, in milliseconds since the Unix epoch. Can be synchronized with current time.

7. RubyFPV SDK Hooks

The plugin implements the full interface defined in ruby_core_plugin.h. Here is exactly what each hook does:

Hook	Our implementation
core_plugin_on_requested_capabilities()	Returns VIDEO_STREAM | DATA_STREAM
core_plugin_get_name()	Returns "FrameTap - RubyFPV Frame Extractor"
core_plugin_get_guid()	Returns "FT9921-RFP44-XT01-FRAME"
core_plugin_get_version()	Returns 1
core_plugin_init()	Creates output dir, opens log.json, spawns ft_udp_thread
core_plugin_uninit()	Sets g_bRunning=false, joins thread, closes log file
core_plugin_on_rx_data()	EMPTY, plugin opens its own UDP socket independently
core_plugin_has_pending_tx_data()	Returns 0, plugin does not transmit data
core_plugin_on_get_video_stream_source_type()	Returns CORE_PLUGIN_VIDEO_STREAM_SOURCE_IP
core_plugin_on_setup_video_stream_source()	Returns 1, reports stream as available
All remaining hooks	Empty stubs, not needed for frame extraction

---

8. Connecting to the Real Wombat

Current state: software simulation only. Not tested on real hardware.

Two source code changes are needed to connect to a real Wombat ground station:

8.1 Change the UDP Port

```
// Current (localhost test)
#define UDP_PORT 7012


// Real Wombat, RubyFPV pipes video to UDP port
#define UDP_PORT port
```

8.2 Accept Packets from Any Source

```
// Current, localhost only
snprintf(url, sizeof(url), "udp://127.0.0.1:%d", UDP_PORT);


// Real Wombat, accept from local interface
```

```
snprintf(url, sizeof(url), "udp:_._._._:%d", UDP_PORT);
```

9. Known Limitations

Limitation	Details	Fix needed
Not tested on real hardware	All testing done with ffmpeg simulation on localhost	Test on real Wombat
Localhost only	URL hardcoded to 127.0.0.1	Accept packets from interface
No runtime frame rate control	SAVE_EVERY_N is a compile-time constant	To add config file or environment variable
No stream reconnection	If UDP stream drops, thread exits permanently	To add reconnect loop with backoff in ft_udp_thread

10. Dependencies

Library	Version confirmed	Purpose
libavformat	61.7.100	Open UDP stream, demux MPEG-TS container
libavcodec	61.19.101	Decode H.264 compressed video to raw YUV frames
libavutil	59.39.100	Frame allocation, pixel format definitions, image utils
libswscale	8.3.100	Convert YUV420p pixel data to RGB24
pthreads	POSIX	Background decoder thread, pthread_create / pthread_join
stb_image_write.h	header-only	Encode RGB24 raw data to JPEG, stbi_write_jpg
ruby_core_plugin.h	from RubyFPV source	Plugin SDK type definitions and hook signatures